Pipeline vs. End-to-End Models, and Simple Ways to Reduce Cascading Errors: A Literature Review

Ifty Rashedul Arefin

Meeting 5 Report

July 9 2026

Abstract

This report answers two questions from Meeting 4. *First:* recent research uses "end-to-end" (E2E) driving models, which are one big neural network. Our project uses a "pipeline" instead, which is three separate models joined in a line: C1 (Perception, our YOLOv11 object detector), C2 (Prediction, our constant-velocity trajectory model), and C3 (Planning, our rule-based IDM planner). Why do we use a pipeline? *Second:* what is a simple first method we can use to reduce cascading errors (an error in one model spreading to the next)? I read recent papers to answer both. The short answer to the first: in real deployed systems, updates happen one part at a time (the perception model gets retrained on new data, the map changes, a sensor is swapped). An E2E model is a single block, so you cannot update just one part; you must retrain the whole thing, which is expensive and can make the model forget what it already knew. A pipeline lets you update the one part that changed and leave the rest, which is how real systems are actually maintained. The short answer to the second: build a small "drift gate" that checks whether a newly retrained C1 (Perception) produces very different outputs than before, *before* we let it into the pipeline.

1 Executive Summary

Task 1: pipeline vs. E2E. E2E models (one big network from sensors to driving plan) are the popular trend today; one survey counts more than 270 papers by 2024 [1]. But the same papers admit three problems:

- E2E models are hard to understand and hard to debug [1, 2].
- You cannot fix one small part; you must retrain the whole model, which is slow and can make the model forget what it already learned [3, 4].
- The test score everyone reports is known to be unreliable [5–7]. (This is the nuScenes *open-loop* score: the model predicts one path for a scene, and we measure how far that path is from the human driver's recorded path. The car never actually drives, so a small distance does not prove the car would drive safely. We use this term throughout the report.)

This matters a lot in real deployment, where updates almost always happen *one part at a time*: a new detector is retrained on fresh data, the map is updated, or a sensor is changed, while the other parts stay fixed. A pipeline has three parts we can update separately, which matches this real-world pattern. An E2E model has only one part, so every small change means retraining everything. So a pipeline is the practical choice, not a weaker one.

Task 2: reducing cascading errors. The papers give three groups of methods:

- *Catch the problem early*, by checking the data passed between models.
- *Retrain carefully*, so the new model does not behave too differently from the old one.
- *Control the process*, by testing the new model quietly before using it.

We recommend one simple first method: a **drift gate** between C1 (Perception) and C2 (Prediction). Before we accept a retrained C1, we compare its outputs to the old C1's outputs using a simple statistics test. If they differ too much, we raise a warning.

2 Recent End-to-End Driving Models

An end-to-end model takes the camera and sensor input and directly outputs the driving plan, all inside one network. Below are the most important papers, chosen by how strong their publication is (top conferences and journals first). The rest are listed in Table 1.

UniAD [8] (CVPR 2023, Best Paper). This is the most famous E2E model. It puts every driving task (finding objects, tracking them, predicting their motion, and planning) into one network, and every part is trained together to help the final plan. *Good:* it was the first "planning-first" unified model and won the best paper award. *Bad:* it is very heavy to train (about 144 GPU-hours) and runs slowly (1.8 frames per second) [9]; if you want to change anything, you must retrain everything. *For us:* this is the main model we compare against.

VAD [10] (ICCV 2023). VAD makes E2E driving faster by describing the scene with simple lines and vectors instead of heavy image grids. *Good:* much faster than UniAD, with similar scores. *Bad:* still one big model you must retrain all at once. *For us:* together with UniAD, it is the model that later "repair" papers try to fix.

PARA-Drive [11] (CVPR 2024). This paper studies how to arrange the parts inside an E2E model. It shows a design where the parts run side by side (in parallel) and can even be turned off at run time. *Good:* lower error and faster than UniAD. *Bad:* the parts still share one trained network, so you still cannot retrain just one part. It also showed that the published UniAD-vs-VAD comparison was not fair, because the two used different test settings. *For us:* good evidence that E2E test scores must be read carefully.

SparseDrive [9] (2024). SparseDrive uses a lighter scene description to make training and running faster. *Good:* it reports that it trains 7 times faster and runs 5 times faster than UniAD. *Bad:* still tested only with open-loop scores. *For us:* its own numbers prove how expensive it is to retrain a full E2E model, which supports our cost argument.

Hydra-MDP [12] (2024, won the CVPR 2024 challenge). This model learns from both human driving and from a rule-based planner, all inside one network. *Good:* it won a large 2024 competition, and it shows rule-based knowledge is still very useful. *Bad:* it hides the rule-based part inside the network, so you can no longer check or update that part by itself. *For us:* an example of why putting everything into one block hurts maintenance.

Survey by Chen et al. [1] (IEEE TPAMI 2024). This is the main survey of E2E driving. It lists the open problems: E2E models are hard to understand, get confused about cause and effect, and are not robust to new conditions. *For us:* the survey is written by E2E supporters, so when they admit these weaknesses, it is strong evidence. Also, the survey never studies retraining policy, which is exactly our topic. That gap is our opportunity.

Codevilla et al. [5] (ECCV 2018). This early paper showed a key fact: a model's offline test score does not always match how well it actually drives. Two models with the same score can drive very differently. *For us:* this is an honest warning we must also apply to our own planning score.

Dauner et al., PDM-Closed [7] (CoRL 2023). A simple rule-based planner won the 2023 nuPlan planning challenge and beat the learned planners. *For us:* this helps in three ways:

- It supports our choice of a simple rule-based planner (C3).
- It shows that "learned planning is better" is not proven.
- It supports how we measure planning.

Dauner et al., NAVSIM [13] (NeurIPS 2024). NAVSIM is a middle ground between two ways of testing. The open-loop score (defined above) never lets the car move. The opposite is full closed-loop simulation, where the car really drives, but that is heavy and slow. NAVSIM sits in between: it takes the model's planned path and rolls it forward for a short time in a light simulation, then scores useful things like how much progress the car made and whether it would have hit something (time to collision). These scores match real driving much better than the open-loop distance. A large 2024 competition (143 teams) used it, and it also found that simple models can match the big flagship models, so extra complexity does not always help. *For us:* this is the stronger test we can move to once we need to say how well our system drives, not just whether it got better or worse after one change.

Zhai et al., "AD-MLP" [6] (2023). This paper showed something surprising: a tiny model using only the car's own past motion (no camera at all) matches perception-based E2E models on the nuScenes open-loop score. *For us:* strong proof that the common test score is weak, so we should report the collision rate too (we do) and compare our numbers before-and-after rather than as absolute values.

Table 1: Other E2E and repair papers (mostly recent preprints). These are the "repair / maintenance" works. They matter because they show a new trend: even E2E research is now trying to fix models part by part, which supports our pipeline idea.

Paper	**Main idea (plain words)**	**Why it matters to us**
CorrectAD [3] (2025)	Finds failures, makes fake training videos, then retrains the *whole* E2E model. Fixes about 62.5% of failures.	Proof that in an E2E model the smallest unit you can retrain is the whole model.
R2SE [4] (2025)	Freezes the big model and trains small add-on parts, because full retraining makes the model forget.	The E2E field is rediscovering modular (part-based) design.
ADReFT [14] (2025)	Repairs driving decisions with a separate add-on module, not by touching the base model.	Same lesson: add parts instead of retraining the monolith.
VLA survey [2] (2025)	Surveys the newest "vision-language-action" driving models. Many split into a slow "thinking" part and a fast "acting" part.	Even the newest models move back toward separate parts.

3 Pipeline vs. End-to-End: Side-by-Side

Table 2 compares the two designs. The last two rows are honest: the one real strength of E2E is that it trains all parts together. And the one real weakness of a pipeline (errors passing between parts) is a real problem we do not hide. But note that this weakness only appears *after* an update, when one part changes and the others do not, which is exactly the situation every deployed system faces during maintenance. So it is a problem worth managing, not a reason to abandon the design that real systems actually use.

Table 2: Pipeline (ours) vs. end-to-end, in simple terms.

Point	**Pipeline (ours)**	**End-to-end**	**Evidence**
Easy to understand	Each step (detections, predictions) can be looked at	One black box, hard to see inside	[1, 2]
Update one part	Can retrain any of the 3 parts alone	Must retrain the whole model	[3, 4]
Easy to debug	Can test each part alone to find the fault	Needs special methods just to locate the fault	[1, 3]
Retraining cost	Retrain only the changed part (about 50 s for our C1)	Retrain everything (UniAD about 144 GPU-hours)	[3, 9]
Robust to new conditions	Each part's behavior can be measured	Sensitivity and forgetting are still open problems	[1, 4]
Grows easily	Add or swap parts one by one	Any change needs a new full training run	[1, 9]
Compute cost	Light parts run on free hardware	Slow to run, days to train	[9, 11]
Data needed	Normal labels; small fine-tunes are possible	Needs very large datasets	[1, 3]
Best overall accuracy	*Weaker:* parts have different goals; some info is lost between them	*Stronger:* all parts trained together	[8, 9]
Trustworthy testing	Each part checked against ground truth	Common open-loop test is unreliable	[5–7]

4 Why We Use a Pipeline

Reason 1: it matches how real systems are actually updated. In practice, a driving system is not retrained all at once. One part is updated when its own data drifts (for example, the perception model is retrained on new-city data), while the other parts stay fixed. This is how companies actually maintain deployed systems: they fix the part that broke, not the whole car's software. A pipeline supports this directly, because you can update one part and leave the rest. An E2E model cannot; its only update action is "retrain the whole thing" [3], which means every small data change forces a full, expensive rebuild. So the pipeline is not a smaller choice; it is the design that matches the real update pattern of deployed systems.

Reason 2: a pipeline lets you find and fix faults; E2E hides them. The E2E side says pipelines are bad because errors pass between parts [1, 9]. But cascading errors do not disappear

in an E2E model; a classic industry paper [15] showed the same tangling still happens inside one big model, it just becomes harder to see. In a real deployed system this visibility is what matters: when something goes wrong, an engineer must be able to point to which part failed and fix that part. A pipeline makes that possible because each part has its own output and its own metric. In an E2E model the fault is buried inside one network, and the only response is to retrain everything.

Reason 3: real maintenance is cheaper with a pipeline. Deployed systems retrain again and again as the world changes: new cities, new seasons, new sensors. With a pipeline you retrain only the part that drifted, which for our detector takes about 50 seconds. With an E2E model every such change forces a full retrain, about 144 GPU-hours [9], and repairing an E2E model can even need generating fake failure data [3] because real failure cases are rare and unsafe to collect. Over the many updates a real system needs across its lifetime, the pipeline is far cheaper to keep running.

Reason 4: even E2E research is moving back to parts. The clearest sign that parts matter for real use is that E2E research itself is adding them back once maintenance becomes the goal: CorrectAD must locate failures before fixing them [3], R2SE trains small add-on modules [4], ADReFT adds a separate repair module [14], and new VLA models split thinking from acting [2]. When the question changes from "highest score on a benchmark" to "how do we keep this running and fix it," the field moves from one block back toward separate parts, which is exactly what a pipeline already is.

Reason 5: an honest note about our own testing. The open-loop score is weak at one job: saying "model A drives better than model B" [5,6]. We do not use it for that job. We never compare our pipeline against another model. We only ask one question: did the planner's error go up or down after we retrained C1, using the same planner on the same scenes? For measuring the effect of one change on one fixed system, the score is fine. If we later want to claim our system drives well in an absolute sense, we will switch to a stronger test like NAVSIM [13].

Open problems (still unsolved by anyone).

1. No paper gives a rule for *which part to update, when, and in what order* in a real driving system [1, 2]. This is a practical maintenance question every operator faces.
2. The E2E "repair" area is very new (2025–2026), and its tests still use the weak open-loop score [3, 4], so we do not yet know if the repairs help real driving.
3. No one has connected software-reliability theory (availability and continuity, the same tools used for other safety-critical systems) to how these driving models are updated and maintained.

5 Ways to Reduce Cascading Errors

A cascading error means: one model gets a little worse, and that makes the next model worse, and so on down the line. Below are the strongest papers, explained simply. The rest are in Table 3.

Sculley et al. [15] (NIPS 2015). This famous industry paper explains the core problem in one rule: in an ML system, changing anything can change everything. When one model is built on top of another model's output, improving one part can quietly hurt the whole system. *For us:* our E1 result (C2, Prediction, got much better, but C3, Planning, got worse) is a clean example of exactly this. This paper only *describes* the problem; it does not give an algorithm.

Table 3: Other cascade-mitigation methods (grouped by type).

Method	Main idea (plain words)	Type	Difficulty
OOD detection [20] (ITSC 2021)	Detect inputs C1 (Perception) has never seen, and trigger a safe fallback.	Catch early	Medium
MOT-CUP [21] (RA-L 2024)	Pass C1's (Perception) uncertainty into tracking; helps most under occlusion.	Catch early	Advanced
Uncertainty in planning [22] (2024)	Planners that ignore prediction uncertainty perform worse.	Catch early	Advanced
System-level view [23] (2024)	A part's quality only matters relative to the part that uses it; a downstream part can even absorb upstream error.	Analysis	Advanced
FastFill [24] (ICLR 2023)	Update the model without recomputing everything downstream (which can take months).	Careful retrain	Medium
Coordinated retrain [15]	Retrain C2 (Prediction) on the new C1's (Perception) output before using it. (Already one of our E2–E7 options.)	Process	Easy–Med
Shadow / gated deploy [16]	Run the new model quietly first; use it only if the system does not get worse. Our isolated-vs-pipeline test is a manual version of this.	Process	Easy

Data validation / TFDV [16] (SysML 2019). Google's method checks the data that flows between steps of a pipeline. It compares the new data to the expected data using simple distance measures, and raises a warning if they differ too much. *Good:* used in industry, cheap, no model change needed. *Limit:* it checks the data only, not the final score. *Difficulty: easy.* This is the model for our recommended baseline.

Backward-compatible training (BCT) [17] (CVPR 2020). This method retrains the new model so its output still "fits" the old downstream model, so you do not have to change the downstream part. Without it, a freshly retrained model can be completely incompatible with the old one. *Limit:* keeping compatibility can slightly lower the new model's own score, and it was designed for image search, so it needs some adaptation for detection. *Difficulty: medium.*

Positive-congruent training [18] (CVPR 2021). When you update a model, some inputs that the old model got right, the new model now gets wrong. These are called "negative flips," and they happen even when the new model is more accurate overall. This method trains the new model to keep agreeing with the old model on those cases. *For us:* this is the same idea as our problem but at the single-example level (better on average, worse on some cases). *Difficulty: medium.* This is our natural second method after the drift gate.

Ivanovic et al. [19] (ICRA 2022). Normally C1 (Perception) passes only its best guess to C2 (Prediction). This paper passes the uncertainty too, so C2 knows how sure C1 is. On a nuScenes pipeline very close to ours, this improved prediction. *For us:* a strong "next step" after the baseline, but it needs changes to how the parts talk to each other, so it is advanced.

6 Recommended First Baseline

The idea: a drift gate between C1 (Perception) and C2 (Prediction). Before we accept a retrained C1 into the pipeline, we compare its outputs to the old C1's outputs on the same fixed set of images. We look at simple things: how many objects it finds per image, how confident it is, which classes it predicts, and where the boxes are. We compare old vs. new with two standard, simple tests. For number-type features (like the confidence values), we use the *Kolmogorov–Smirnov test*, which measures the biggest gap between the two value distributions and gives a single number for how far apart they are. For category-type features (like which object classes appear), we use the *Jensen–Shannon distance*, which measures how different the two category mixes are on a 0-to-1 scale (0 means identical, 1 means completely different). This is the same idea as *Google's data validation* [16], a widely used industry tool that watches the data flowing between the steps of a machine learning pipeline and raises a warning when today's data looks statistically different from what the system expects. If the difference is too large, we raise a "cascade risk" warning and do not blindly use the new C1. Figure 1 shows the idea.

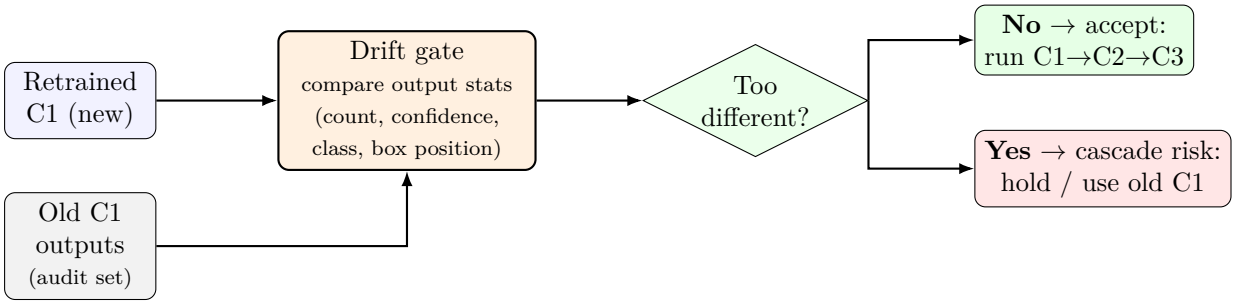

Figure 1: The drift gate. Each time C1 (Perception) is retrained, we compare the *new* C1's output statistics against the *old* C1's on a fixed set of images. If they are close, the new C1 is accepted into the pipeline. If they differ too much, we flag a cascade risk and hold the new C1 (or keep the old one) instead of letting the change flow silently into C2 and C3.

Why this one first.

1. **Easy and free.** About 100 lines of Python over files our pipeline already produces. No GPU, so it fits our free Kaggle setup.
2. **It would have caught our E1 case.** After the Singapore fine-tune, C1's (Perception) recall dropped from 0.087 to 0.048 and its outputs clearly shifted. So we can test the gate on our existing v22 results right away, before running anything new.
3. **It fits our theory.** The gate gives an early warning signal, which is exactly the kind of number the SMP model needs to estimate risk. It is also the simplest version of the "entanglement forecaster" idea from our original proposal.
4. **It is supported by the literature.** Checking the data between parts is the recommended control for this problem [15, 16], and "better on average but worse on some cases" is the exact failure it catches [18].
5. **It has a clear next step.** After the gate, method B2 (positive-congruent training) can *reduce* the drift the gate *detects*. Together they make a nice detect-then-fix pair for a future meeting.

Honest limits. We should be clear about what this simple gate cannot do, so it is not oversold.

- **It sees change, not harm.** The gate only tells us that the new C1's outputs are *different* from the old one's. It does not tell us whether that difference makes the final driving *worse*.

In fact, in our own E1 result the outputs changed a lot, yet C2 (Prediction) actually got *better*. So a warning from the gate means "look closely," not "this is bad."

- **The warning line has to be chosen by hand.** The gate fires when the difference passes a threshold, but there is no obvious correct value for that threshold. Set it too low and it cries wolf on harmless changes; set it too high and it misses real ones. We will have to tune it by trying values and seeing which ones catch the cases we care about.
- **We have too little data to trust the numbers yet.** With only 3 Singapore test scenes, we cannot yet measure how often the gate gives a false alarm or misses a real problem. Those rates need many more scenes before we can report them with confidence.

The good news is that each of these limits is itself something we can *measure* once we run the gate on real data, so this baseline is a genuine first experiment, not just a box to tick.

References

[1] L. Chen, P. Wu, K. Chitta, B. Jaeger, A. Geiger, and H. Li, "End-to-end autonomous driving: Challenges and frontiers," *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2024, arXiv:2306.16927.

[2] T. Hu *et al.*, "Vision-language-action models for autonomous driving: Past, present, and future," *arXiv preprint arXiv:2512.16760*, 2025.

[3] E. Ma, L. Zhou, T. Tang, J. Zhang, J. Jiang, Z. Zhang, D. Han, K. Zhan, X. Zhang, X. Lang, H. Sun, X. Zhou, D. Lin, and K. Yu, "Correctad: A self-correcting agentic system to improve end-to-end planning in autonomous driving," *arXiv preprint arXiv:2511.13297*, 2025.

[4] H. Liu, T. Li, H. Yang, L. Chen, C. Wang, K. Guo, H. Tian, H. Li, H. Li, and C. Lv, "Reinforced refinement with self-aware expansion for end-to-end autonomous driving," *arXiv preprint arXiv:2506.09800*, 2025.

[5] F. Codevilla, A. M. López, V. Koltun, and A. Dosovitskiy, "On offline evaluation of vision-based driving models," in *European Conference on Computer Vision (ECCV)*, 2018, arXiv:1809.04843.

[6] J.-T. Zhai, Z. Feng, J. Du, Y. Mao, J.-J. Liu, Z. Tan, Y. Zhang, X. Ye, and J. Wang, "Rethinking the open-loop evaluation of end-to-end autonomous driving in nuscenes," *arXiv preprint arXiv:2305.10430*, 2023.

[7] D. Dauner, M. Hallgarten, A. Geiger, and K. Chitta, "Parting with misconceptions about learning-based vehicle motion planning," in *Conference on Robot Learning (CoRL)*, 2023, won 2023 nuPlan challenge; arXiv:2306.07962.

[8] Y. Hu, J. Yang, L. Chen, K. Li, C. Sima, X. Zhu, S. Chai, S. Du, T. Lin, W. Wang, L. Lu, X. Jia, Q. Liu, J. Dai, Y. Qiao, and H. Li, "Planning-oriented autonomous driving," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, best Paper Award; arXiv:2212.10156.

[9] W. Sun, X. Lin, Y. Shi, C. Zhang, H. Wu, and S. Zheng, "Sparsedrive: End-to-end autonomous driving via sparse scene representation," *arXiv preprint arXiv:2405.19620*, 2024.

[10] B. Jiang, S. Chen, Q. Xu, B. Liao, J. Chen, H. Zhou, Q. Zhang, W. Liu, C. Huang, and X. Wang, "Vad: Vectorized scene representation for efficient autonomous driving," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, arXiv:2303.12077.

[11] X. Weng, B. Ivanovic, Y. Wang, Y. Wang, and M. Pavone, "Para-drive: Parallelized architecture for real-time autonomous driving," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[12] Z. Li, K. Li, S. Wang, S. Lan, Z. Yu, Y. Ji, Z. Li, Z. Zhu, J. Kautz, Z. Wu, Y.-G. Jiang, and J. M. Alvarez, "Hydra-mdp: End-to-end multimodal planning with multi-target hydra-distillation," *arXiv preprint arXiv:2406.06978*, 2024, 1st place, NAVSIM track, CVPR 2024 Autonomous Grand Challenge.

[13] D. Dauner, M. Hallgarten, T. Li, X. Weng, Z. Huang, Z. Yang, H. Li, I. Gilitschenski, B. Ivanovic, M. Pavone, A. Geiger, and K. Chitta, "Navsim: Data-driven non-reactive autonomous vehicle simulation and benchmarking," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024, arXiv:2406.15349.

[14] M. Cheng, X. Xie, R. Wang, Y. Zhou, and M. Hu, "Adreft: Adaptive decision repair for safe autonomous driving via reinforcement fine-tuning," *arXiv preprint arXiv:2506.23960*, 2025.

[15] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems (NIPS)*, vol. 28, 2015, pp. 2503–2511.

[16] E. Breck, N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, "Data validation for machine learning," in *Proceedings of Machine Learning and Systems (SysML/MLSys)*, vol. 1, 2019.

[17] Y. Shen, Y. Xiong, W. Xia, and S. Soatto, "Towards backward-compatible representation learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 6368–6377, arXiv:2003.11942.

[18] S. Yan, Y. Xiong, K. Kundu, S. Yang, S. Deng, M. Wang, W. Xia, and S. Soatto, "Positive-congruent training: Towards regression-free model updates," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 14 299–14 308, arXiv:2011.09161.

[19] B. Ivanovic, Y. Lin, S. Shrivastava, P. Chakravarty, and M. Pavone, "Propagating state uncertainty through trajectory forecasting," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2022, arXiv:2110.03267.

[20] J. Nitsch, M. Itkina, R. Senanayake, J. Nieto, M. Schmidt, R. Siegwart, M. J. Kochenderfer, and C. Cadena, "Out-of-distribution detection for automotive perception," in *2021 IEEE International Intelligent Transportation Systems Conference (ITSC)*, 2021, pp. 2938–2943.

[21] S. Su, S. Han, Y. Li, Z. Zhang, C. Feng, C. Ding, and F. Miao, "Collaborative multi-object tracking with conformal uncertainty propagation," *IEEE Robotics and Automation Letters (RA-L)*, vol. 9, no. 4, 2024, arXiv:2303.14346.

[22] W. Shao, J. Xu, Z. Cao, H. Wang, and J. Li, "Uncertainty-aware prediction and application in planning for autonomous driving: Definitions, methods, and comparison," *arXiv preprint arXiv:2403.02297*, 2024.

[23] S. Deglurkar, H. Shen, A. Muthali, M. Pavone, D. Margineantu, P. Karkus, B. Ivanovic, and C. J. Tomlin, "System-level analysis of module uncertainty quantification in the autonomy pipeline," *arXiv preprint arXiv:2410.12019*, 2024.

[24] F. Jaeckle, F. Faghri, A. Farhadi, O. Tuzel, and H. Pouransari, "Fastfill: Efficient compatible model update," in *International Conference on Learning Representations (ICLR)*, 2023, arXiv:2303.04766.